

On the Parallel Distributed List Update Problem*

Hsin-Lung Wu

Department of Computer Science and Information Engineering
National Taipei University, New Taipei City, Taiwan
hsinlung@mail.ntpu.edu.tw

Abstract

We study a variant of the list update problem in a distributed environment which is called parallel distributed list update problem (PDLUP). In this setting, the set of items is partitioned into k lists $L_1, \dots, L_k$ and each list L_i is maintained by a processor i . Each processor can communicate with others and can move its item to any position of different list by incurring a communication cost. The goal is to design an online algorithm which minimizes the sum of the parallel search cost for PDLUP when given a request sequence. We propose a family of online algorithms called move-to-front-of-any-list-like algorithms (MTFAL-like algorithms) and show that they are $2k$ -competitive online algorithms for PDLUP. We also show that some MTFAL-like algorithms which are called fixed MTFAL-like algorithms have their competitive ratio at least $2k$. Thus for these algorithm, $2k$ is the best competitive ratio. Finally, we show that every deterministic online algorithm for PDLUP has competitive ratio at least 2.

1 Introduction

The list update problem is one of the famous topics in the area of online problems. The problem is described as follows. Given an unsorted linear list $\mathcal{L}$ of n distinct items and a finite requested sequence σ of size m , where each request is either an access, insert or delete operation on an items in the list $\mathcal{L}$. One can see Insert and Delete operations as a special case of access operation. So we assume that only access operations are performed and refer the list update problem as the list accessing problem. A request is serviced if an

access operation of the requested item in the list $\mathcal{L}$ is performed by incurring some access cost. After accessing the requested item, one may rearrange the items in the list $\mathcal{L}$ by incurring extra reorganizing cost before serving the next request. The goal of the list update problem is to minimize the total access cost and reorganizing cost when we serving a given request sequence σ on the list $\mathcal{L}$.

To minimize the total cost for a given request sequence σ , it is unavoidable to know the information of the request sequence σ . Thus, the optimal solution of the list update problem is usually an off-line solution. However, the assumption that an algorithm knows the request sequence σ is often unrealistic in many practical applications. There are many algorithmic problems which are online in the sense that the input is only partial given and some relevant input data is available in the future and is not accessible at present. An online algorithm should produce output without the whole input information. One way to evaluate an online algorithm is the so-called competitive analysis where an online algorithm is compared to an optimal off-line algorithm. Given a request sequence σ , let $C_A(\sigma)$ and $C_{OPT}(\sigma)$ denote the cost incurred by the online algorithm A and an optimal off-line algorithm OPT in processing σ , respectively. We say the online algorithm A is c -competitive if there exists a nonnegative integer constant α such that $C_A(\sigma) \leq c \cdot C_{OPT}(\sigma) + \alpha$ for all request sequence σ . The value c is usually called the competitive ratio of the online algorithm A .

One of the online algorithms for solving list update problem is the so-called move-to-front (MTF) algorithm. The MTF algorithm moves the last accessed item to the front of the list. In their seminal paper [3], Sleator and Tarjan show that MTF is 2-competitive and no deterministic on-line algorithm can perform better than MTF. List update algorithms have many applications such as computing point maxima and convex hulls

*This work was supported in part by the National Science Council of Taiwan under contract NSC-99-2221-E-305-009-MY3.

[2] and data compression [1].

In this paper, we consider a variant of the list update problem in a distributed environment which is called parallel distributed list update problem (**PDLUP**). In this setting, the set of items is partitioned into k lists $L_1, \dots, L_k$ and each list L_i is maintained by a processor i . Each processor can communicate with others and can move its item to any position of different list by incurring a communication cost. Definitions of the cost of accessing an item in PDLUP can be made by a parallel sense. This sense captures parallel processing search time. In this setting, the cost is defined as a combination of a list searching cost and a communication cost where we call it **parallel cost**. We will give formal definitions in the next section.

1.1 Our contribution

In this paper, we propose a family of online algorithms called Move-To-the-Front-of-Any-List-like algorithms (**MTFAL**-like algorithms) to solve the parallel distributed list update problem. A MTFAL-like algorithm always moves the accessed item from its current list to the front of the some list which may be the original list. We show that every MTFAL-like algorithm is $2k$ -competitive online algorithm in the parallel cost model. In addition, we show that, for some particular MTFAL-like algorithms, the competitive ratio $2k$ cannot be improved. Next, we also show that no deterministic online algorithm for PDLUP can obtain a competitive ratio less than 2. Note that this leaves a gap of competitive ratio between 2 and $2k$.

1.2 Organization of the paper

The remaining part of the paper is organized as follows. In Section 2, we give formal definitions for the parallel distributed list update problem and its cost model. In addition, we describe the move-to-front-of-Any-list-like algorithms (MTFAL-like algorithms). In Section 3, we show the competitive analysis for MTFAL-like algorithms. In Section 4, we show that no deterministic online algorithm can have competitive ratio less than 2. Finally, we give a conclusion in Section 5.

2 Preliminaries

2.1 Parallel Distributed Lists

A set of n items is partitioned into k parts. Items in part i are stored by a list L_i for $1 \leq i \leq k$. Each list L_i is maintained by a processor i . For each processor i , we assume that the cost of each searching step in the list L_i is one. Each processor can communicate with others and can move its item to any position of different list by incurring an extra communication cost c .

2.2 Cost Models

Algorithms for parallel distributed list update problem incur two types of costs on each request: the parallel list access cost of searching in the lists and the communication cost when broadcasting information from one processor to others. First, we assume that the communication cost is always a fixed constant c per transmitting round. Second, let m_i denote the number of items in the i -th list which are scanned before locating the requested item. The parallel searching cost is defined by sum of the minimum value $\min\{m_1, \dots, m_k\}$ and the cost of making any rearrangements to all lists. Note that the parallel searching cost captures the parallel time for searching the accessed item in a distributed system.

Algorithms are allowed to rearrange the lists during each access by using free and paid exchanges. Algorithms can use free exchanges to move an item on a list closer to the front of the list or to the front of the other list at no extra cost. Any other move must consist in exchanges of consecutive items in each list. Each of these exchanges is called a paid exchange and has unit cost. In the case that a free exchange that transfers an item to the other list, the communication cost c is still required.

A request sequence of length m is a sequence $\sigma = \langle a_1, a_2, \dots, a_m \rangle$ where a_i is the i -th request item. An algorithm M , on each request, starts a search task that is executed in parallel by k processors. Each processor searches a_i on its own list until one of the processors locates the item a_i . When some processor finds the requested item a_i , it notifies others to stop searching and this incurs a communication cost c .

The parallel distributed list update problem (PDLUP) is to minimize the total parallel cost and reorganizing cost when we serving a given request sequence σ . We usually require an online algorithm for solving PDLUP.

2.3 Move-To-the-Front-of-Any-List-like Algorithm (MTFAL-like Algorithms)

We consider the generalized version of the well-known move-to-front online algorithm for list update problem [3]. A Move-To-the-Front-of-Any-List-like algorithm (MTFAL-like algorithm) always moves the accessed item to the front of some list which may be the original list. For example, Move-To-the-Front-of-the-Shortest-List algorithm (MTFSL algorithm) which always moves the accessed item to the front of the shortest list is an example of MTFAL-like algorithms. For the MTFSL algorithm, if there are at least two shortest lists, we choose the shortest list which has the minimum lexicographic order among those shortest lists. We illustrate how MTFSL works by the following examples. Suppose the set of items is initially partitioned into three lists shown in the following table.

List 1	1	3	7	9
List 2	2	4	6	10
List 3	8	5		

After accessed item 4, MTFSL moves item 4 to the front of List 3 which is the shortest list. In this case, the parallel cost of MTFSL is $2 + c$. The result is shown in the following table.

List 1	1	3	7	9
List 2	2	6	10	
List 3	4	8	5	

Then, after accessed item 9, MTFSL moves item 9 to the front of List 2 which has the minimum lexicographic order among List 2 and List 3. In this case, the parallel cost of MTFSL is $4 + c$. The result is shown in the following table.

List 1	1	3	7	
List 2	9	2	6	10
List 3	4	8	5	

3 Competitive Analysis for MTFAL-like algorithms

In this section, we show that MTFSL is $2k$ -competitive in the parallel cost model. Before proving the statements, we need some definitions given as follows. Given a list L and an item x on it, let $\mathbf{rank}_L(x)$ be the rank of x in L .

Definition 1. Let $S \subset \mathbb{R}$ be a set of size n and D be a family of lists $\{L_1, \dots, L_k\}$ which form a partition of S . $<_D$ is a partial relation on $S \times S$ defined as follows. Given $x, y \in L_j$ for some j , then we define $x <_D y$ if $\mathbf{rank}_{L_j}(x) < \mathbf{rank}_{L_j}(y)$. In particular, we define the partial relation $\leq_D$ on $S \times S$ as follows: given $x, y \in L_j$ for some j , $x \leq_D y$ if $\mathbf{rank}_{L_j}(x) \leq \mathbf{rank}_{L_j}(y)$.

Definition 2. $\prec_D$ is a total relation on $S \times S$ defined as follows: for any $x, y \in S$, $x \prec_D y$ if $\mathbf{rank}_{L_{j_x}}(x) < \mathbf{rank}_{L_{j_y}}(y)$ where $x \in L_{j_1}$ and $y \in L_{j_2}$ for some $j_1, j_2 \in \{1, \dots, k\}$. In particular, we define the total relation $\preceq_D$ on $S \times S$ as follows: for any $x, y \in S$, $x \preceq_D y$ if $\mathbf{rank}_{L_{j_1}}(x) \leq \mathbf{rank}_{L_{j_2}}(y)$ where $x \in L_{j_1}$ and $y \in L_{j_2}$ for some $j_1, j_2 \in \{1, \dots, k\}$.

Intuitively, given $x, y \in S$, if $x \prec_D y$, then the length of the partial list from the front to x is less than that of the partial list from the front to y .

Theorem 1. Every MTFAL-like algorithm is $2k$ -competitive in parallel cost model where k is the number of processors in the distributed system.

Proof. Let F be a MTFAL-like algorithm. Our proof is carried out via an amortized analysis of a particular potential function Φ defined later. Let D_i be F 's family of lists $\{L_{i1}, \dots, L_{ik}\}$ after the i -th access. Let D_0 be any given initial set of lists. Note that $\{L_{i1}, \dots, L_{ik}\}$ form a partition of S for each i . Let OPT be the optimal off-line algorithm. Let D_i^* be OPT 's family of lists $\{L_{i1}^*, \dots, L_{ik}^*\}$ after the i -th access. We assume that $D_0^* = D_0$ and D_0 is a set of k lists of the same length. Let c_i be F 's parallel cost for the i -th request and c_i^* be OPT 's parallel cost for the i -th request. If the $(i+1)$ -st accessed item is x and x is in the list L_{ij} , then $c_{i+1} = \mathbf{rank}_{L_{ij}}(x) + c$. For OPT , if x is in the list L_{ij}^* , then $c_{i+1}^* = \mathbf{rank}_{L_{ij}^*}(x) + c + t_{i+1}$ where t_{i+1} is the number of paid exchanges that OPT performs. The potential function Φ that maps each data structure D_i to a real value is defined as follows:

$$\Phi(D_i) = |\{(x, y) : x <_{D_i} y \text{ \& } x \succeq_{D_i^*} y\}|.$$

Suppose the length of the request sequence σ is m . The function Φ has the following properties:

- $\Phi(D_0) = 0$ since $D_0 = D_0^*$.
- $\Phi(D_i) \geq 0$ for any $i \in \{0, 1, \dots, m\}$.

Suppose that x is the $(i+1)$ -st requested item. We define four sets:

- $A = \{y \in S : y <_{D_i} x \text{ \& } y \prec_{D_i^*} x\},$
- $B = \{y \in S : y <_{D_i} x \text{ \& } y \succeq_{D_i^*} x\},$
- $C = \{y \in S : y \geq_{D_i} x \text{ \& } y \prec_{D_i^*} x\},$
- $D = \{y \in S : y \geq_{D_i} x \text{ \& } y \succeq_{D_i^*} x\}.$

Suppose that x is one of items in the list L_{ij} for some $j \in \{1, \dots, k\}$ and one of items in the list $L_{ij'}^*$ for some $j' \in \{1, \dots, k\}$. Clearly, we have

- $\text{rank}_{L_{ij}}(x) = |A \cup B| + 1 = |A| + |B| + 1,$ and
- $|A| \leq |A \cup C| \leq k(\text{rank}_{L_{ij'}^*}(x) - 1).$

Let $L_{i\tilde{j}}$ be the list into which F will move x . Suppose x is not in the list $L_{i\tilde{j}}$. When F moves x to $L_{i\tilde{j}}$, it destroys $|B| + |C|$ inversions and creates at most $k(\text{rank}_{L_{ij'}^*}(x) - 1)$ inversions. Each transpose by OPT creates at most k inversion. In this case, we have

$$\begin{aligned} & \Phi(D_{i+1}) - \Phi(D_i) \\ & \leq k(\text{rank}_{L_{ij'}^*}(x) - 1) - |B| - |C| + kt_{i+1}. \end{aligned}$$

On the other hand, suppose that x is in the list $L_{i\tilde{j}}$. When F moves x to $L_{i\tilde{j}}$, it destroys $|B|$ inversions and creates at most $|A| \leq k(\text{rank}_{L_{ij'}^*}(x) - 1)$ inversions. Each transpose by OPT creates at most k inversion. In this case, we have

$$\Phi(D_{i+1}) - \Phi(D_i) \leq k(\text{rank}_{L_{ij'}^*}(x) - 1) - |B| + kt_{i+1}.$$

In both cases, the amortized cost for the $(i+1)$ -st operation of F with respect to Φ is

$$\begin{aligned} & \hat{c}_{i+1} \\ & = c_{i+1} + \Phi(D_{i+1}) - \Phi(D_i) \\ & \leq \text{rank}_{L_{ij}}(x) + c + k(\text{rank}_{L_{ij'}^*}(x) - 1) \\ & \quad - |B| + kt_{i+1} \\ & = |A| + |B| + 1 + c + k(\text{rank}_{L_{ij'}^*}(x) - 1) \\ & \quad - |B| + kt_{i+1} \\ & = 1 + c + |A| + k(\text{rank}_{L_{ij'}^*}(x) - 1 + t_{i+1}) \\ & \leq 1 + c + k(\text{rank}_{L_{ij'}^*}(x) - 1) \\ & \quad + k(\text{rank}_{L_{ij'}^*}(x) - 1 + t_{i+1}) \\ & = 1 + c - 2k - kt_i + 2k(\text{rank}_{L_{ij'}^*}(x) + t_{i+1}) \\ & \leq 2k(\text{rank}_{L_{ij'}^*}(x) + t_{i+1} + c) + (1 - 2k)(1 + c) \\ & \leq 2kc_{i+1}^*. \end{aligned}$$

Thus, for any access sequence σ , we have

$$\begin{aligned} C_{MTFSL}(\sigma) & = \sum_{i=1}^m c_i \\ & = \sum_{i=1}^m (\hat{c}_i + \Phi(D_{i-1}) - \Phi(D_i)) \\ & = \sum_{i=1}^m \hat{c}_i + \Phi(D_0) - \Phi(D_m) \\ & \leq \sum_{i=1}^m \hat{c}_i \\ & \leq \sum_{i=1}^m 2kc_i^* \\ & \leq 2k \cdot C_{OPT}(\sigma). \end{aligned}$$

□

3.1 Lower Bounds for some particular MTFAL-Like algorithms

A MTFAL-like algorithm A is *fixed* if there exists some list L_i of A with following requirement: for any request sequence consisting of items from L_i only, A only makes free or paid exchanges on the list L_i . We show that, for any fixed MTFAL-like algorithms, the ratio $2k$ cannot be improved.

Theorem 2. *Let A be any fixed MTFAL-like algorithm. For any constant $\varepsilon > 0$, there exists an accessed sequence σ and an off-line algorithm M such that*

$$C_A(\sigma) \geq (2k - \varepsilon)C_M(\sigma).$$

Proof. We assume that the communication cost c is zero and the length t of the shortest list is an integer which is divided by k and $t \gg k$. Since A is fixed, there exists some list L_i of A such that A only makes free or paid exchanges on the list L_i . Without loss of generality, assume that L_i is as $(t, t-1, t-2, \dots, 2, 1)$. The access sequence σ_1 is defined as $\langle 1, 2, \dots, i, \dots, t \rangle$. First of all, we have

$$C_A(\sigma_1) = t^2.$$

On the other hand, we consider the following algorithm M which knows the access sequence σ_1 . M rearranges its collection of lists as the following family by incurring a fixed cost B :

List 1	1	$k+1$	$2k+1$	...	$(t-k+1)$	...
List 2	2	$k+2$	$2k+2$	...	$(t-k+2)$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
List k	k	$2k$	$3k$	...	t	...

After arranging the lists, the cost of M is

$$C_M(\sigma_1) = B + k \cdot \sum_{i=1}^{t/k} i = B + \frac{t(\frac{t}{k} + 1)}{2}.$$

Now, consider the access sequence σ which is d copies of σ_1 , that is,

$$\sigma = \underbrace{\sigma_1 \circ \dots \circ \sigma_1}_d.$$

Then we have $C_A(\sigma) = dt^2$ and $C_M(\sigma) = B + \frac{dt(\frac{t}{k} + 1)}{2}$. Thus, The competitive ratio is

$$\frac{C_A(\sigma)}{C_M(\sigma)} = \frac{2dkt^2}{2kB + dt^2 + dtk}$$

which is close to $\frac{2kt}{t+k}$ as d is sufficiently large. Since $t \gg k$, we must have the ratio $\frac{C_A(\sigma)}{C_M(\sigma)}$ is greater than $2k - \varepsilon$. $\square$

Note that the MTFSL algorithm is a fixed MTFAL-like algorithm since, for any request sequence consisting of items from the shortest list of A , A only makes free or paid exchanges on its shortest list.

Corollary 1. *For any constant $\varepsilon > 0$, there exists an accessed sequence σ and an off-line algorithm M such that*

$$C_{MTFSL}(\sigma) \geq (2k - \varepsilon)C_M(\sigma).$$

4 Lower Bounds for Parallel Distributed List Update Problem

We generalize the lower bounds for any online algorithm for parallel distributed list update problem.

Theorem 3. *If A is c -competitive online algorithm for parallel distributed list update problem, then $c \geq 2$.*

Proof. We construct a request sequence that consist of accessed items only. Each requested item is made to the item which is stored at the last position in A 's longest list. On a request sequence σ of length m obtained in this way, A incurs a cost of $C_A(\sigma) \geq mn/k$. Let M be the optimal static off-line algorithm constructed as follows: M first prepares lists as shown in the following table.

List 1	Item 1	Item $k + 1$	Item $2k + 1$	...	...
List 2	Item 2	Item $k + 2$	Item $2k + 2$	...	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
List k	Item k	Item $2k$	Item $3k$	...	...

Then M sorts the items in the sequence σ in order of non-increasing request frequencies and puts them in the item order shown in the above table. Finally, M services σ without making any further exchanges. When M rearranges its lists, it incurs a cost of at most $n(n + 1)/2$. In addition, the requests in σ can be serviced at a total cost of at most $m(n/k + 1)/2$. Hence,

$$C_M(\sigma) \leq \frac{n(n + 1)}{2} + \frac{mn + mk}{2k}.$$

If $m \gg n$ and $n \gg k$, $n(n + 1)/2$ and k/n can be neglected and for any constant $\varepsilon > 0$

$$\frac{C_A(\sigma)}{C_M(\sigma)} \geq \frac{mn/k}{\frac{n(n+1)}{2} + \frac{mn+mk}{2k}} \geq 2 - \varepsilon.$$

This concludes that the competitive ratio c must be at least 2. $\square$

5 Conclusion

In this paper, we study the parallel distributed list update problem. We show that the move-to-front-of-any-list-like algorithms (MTFAL-like algorithms) is $2k$ -competitive online algorithm for the parallel distributed list update problem (PDLUP). We also show that, for fixed MTFAL-like algorithms, their competitive ratio is at least $2k$. Thus for these algorithm, $2k$ is the best competitive ratio. We also show that every deterministic online algorithm for PDLUP has competitive ratio at least 2. It is interesting to study whether there is a deterministic online algorithm which has competitive ratio less than $2k$. We leave it as an open problem.

References

- [1] M. Burrows and David Wheeler. A block-sorting lossless data compression algorithm. *Technical Report 124, Digital Equipment Corporation*, 1994.
- [2] Jon Bentley, Kenneth Clarkson, and David Levine. Fast linear expected-time algorithms

for computing maxima and convex hulls. In *Proceedings of the first annual ACM-SIAM symposium on Discrete algorithms, SODA '90*, pages 179–187, 1990.

- [3] Daniel Sleator and Robert Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2), pages 202–208, 1985.